

In this session, we will be examining the fourth control structure - Functions

Functions

Contents:

1. [What are Functions](#)
2. [Defining a Function](#)
 - Header
 - Body
 - Arguments
 - Return value
3. [Calling a function](#)
4. [More function definitions](#)
 - Function does not take a value and does not return a value
 - Function that takes one or more value and does not return a value
 - Function that take one or more value and return a value
5. [Variable Scope](#)
6. [Docstring](#)
7. [Lambda expression](#)
8. [Summary](#)

What are Functions

Definition and Purpose

Functions (also known as procedures, modules, methods, or subroutines in other languages) are named blocks of code designed to perform specific tasks. They allow you to:

- Reuse code without rewriting it
- Break down complex problems into manageable pieces
- Improve readability and maintainability
- Facilitate testing and debugging

Key Characteristics

- Functions may accept inputs (arguments)
- Functions may return outputs (return values)
- Functions have their own local scope for variables

Lecture 05

- Functions promote modular programming

A function should perform a single task and perform it really well. If you are not sure that the intended behavior should be implemented in one or more functions, then it is better to err on the side of too many functions. That is because a function is a modular piece of code, and the smaller your functions are, the easier it is to compose (assembly) them together to create more complex behaviours.

Defining a function

Basic Syntax

```
def function_name(parameters):  
    '''docstring''' # Optional documentation  
    # Function body  
    return value # Optional return statement
```

There are two parts of a function:

1. The header (the first line)
There are at least four parts in the header
 - **def keyword:** Signals a function definition.
 - **Function name:** Should be descriptive and follow naming conventions.
 - **Parenthesis:** Contains parameters (optional).
 - **Colon:** Ends the header.
2. The body which is the rest of the definition
The body must contain at least one python statement which must be indented one-level.

The statements in the body is indented one-level

```
#this function does not do anything  
def foo(): #the header  
    pass #the body  
  
# The pass keyword is a nothing statement that is  
# used whenever a statement is needed and you don't have one.  
# In the above case it substitutes for the body of the function,  
# you may also use it to represent the True or the False block of an  
# if-state or the body of a loop.
```

Calling the function

To call or invoke a function, use its name followed by a pair of parenthesis.

```
foo()  
#Function that does not take any input and does not return a value  
def foo():  
    print('\x1B[31mHello world')  
  
foo() # Call the function to execute it  
Hello world
```

Lecture 05

Functions that does not take any argument

These are the easiest types of functions to work with because of its simplicity in usage. However, they are not too flexible.

```
#Function that does not take any input and does not return a value
def my_info():
    print(''
Narendra Pershad
Instructor, Centennial
Toronto, Canada
'')

my_info() # Call the function to execute it
Narendra Pershad
Instructor, Centennial
Toronto, Canada
```

Functions that does not return a value

These types of functions are more useful and versatile because they callers (the code that invokes them) can interact with them by sending different value to them.

```
#Function that take a single argument and still does not return a value
def foo(name):
    print(f'Hello {name}')

#to invoke the above method you must supply EXACTLY one argument
foo('Hao')
# foo() #this does not work
# foo('arben', 'tapia') #this does not work

#Function that takes two arguments and does not return a value
def foo(first, last):
    print( f'{first} {last} ')

#to invoke the above method you must supply EXACTLY two arguments
foo('Hao', 'Lac')
foo('Arben', 'Tapia')
# foo() #this does not work, missing arguments
Hello Hao
Hao Lac
Arben Tapia
```

Functions that return a value

The return keyword is used to send a value back to the caller. This allows you to use the function's result in assignments, expressions, or other function calls.

```
#Function that takes two arguments and return a value
```

Lecture 05

```
def foo(first, last):  
    return f'{first} {last}'  
  
#to invoke the above method you must supply EXACTLY two arguments  
full = foo('Hao', 'Lac')  
print(full)  
  
# alternate way of using foo()  
print(foo('arben', 'tapia'))  
# print(foo()) #this does not work, missing arguments  
Hao Lac  
arben tapia  
# a function calling another function  
def vol(length, width, height):  
    return length * width * height  
  
def mass(l, w, h, den):  
    return vol(l, w, h) * den  
  
l, w, h, d = 2, 3, 4, 10_000  
  
print(f'Length: {l}, width: {w}, height: {h}, density: {d:}, -> mass:  
{mass(l, w, h, d):,}')  
Length: 2, width: 3, height: 4, density: 10,000 -> mass: 240,000
```

All functions return a value

Even if there is no explicit return statement.

```
# function that explicitly returns a value  
def foo(x, y):  
    return x + y  
  
print(foo(1, 2)) # prints 3  
  
# a function that does not explicitly return a value  
def foo(x, y):  
    result = x + y  
  
print(foo(1, 2)) # prints None because the function does not return a value  
3  
None
```

Return values vs. output to screen

Return value is identified by the `return` keyword followed by the value (can be of any datatype that we have covered so far or even a user-defined types) to be returned. This value can be captured by assigning to a variable for future use.

In programming it is important to distinguish between return value and output to the screen.

Output to the screen is achieved via the `print()` function.

Lecture 05

Variable scope

```
a = b = c = 1                                #global scope
def foo():
    global a                                #will use the global variable a
    a, b = 2, 2                            #b is local
    print(f'{a} {b} {c}')
    # local variable b is destroyed when the function is completed

foo()                                       #Prints 2 2 1
print(f'{a} {b} {c}')                   #Prints 2 1 1
2 2 1
2 1 1
```

Default arguments

If the value of one of the arguments to a function is normally unchanged, then you can simplify the calling of the function by setting a default value for that particular parameter. This default **MUST** positioned at the right of the list of arguments i.e. Default argument always follow non-default values.

The `split()` and `print()` methods have default argument of ' ' and '\n' respectively.

```
def greet(name, greeting = 'Hi'):
    print(f'{greeting} {name}')

greet('Ilia')                               #second argument is not explicitly
given, so use it default value
greet('Hao', 'Hello')                     #do not use the default second argument
Hi Ilia
Hello Hao
def calculate_cost(price, tax = 0.13):
    return price + (price * tax)

print(calculate_cost(10))                   #second argument is not explicitly
given, so use it default value
print(calculate_cost(10, 0.2))              #specify both arguments
print(calculate_cost(tax=0.15, price=20))   # named arguments, order
does not matter

# def greet(name = 'Narendra', greeting):    # this does not work, default
argument must be at the end
#     print(f'{greeting} {name}')
11.3
12.0
23.0
```

Working with variable number of arguments

```
def foo(var, *args, **d_args):
    print(f'    var: {var}')
```

Lecture 05

```
print(f'  args: {args}')
print(f'd_args: {d_args}')
print('first call')
foo(1, x=3, y=4)

print('\nsecond call')
foo(1, 2, 3, 4, 5, x = 6, y = 7, z = 8)
first call
  var: 1
  args: ()
d_args: {'x': 3, 'y': 4}

second call
  var: 1
  args: (2, 3, 4, 5)
d_args: {'x': 6, 'y': 7, 'z': 8}
```

Using typing annotation in functions

Python is a dynamically language this means that it does not require formal variable declaration; the variable type is inferred by the interpreter when a value is assigned to it. This makes the language more flexible, but the IDE needs to work harder to provide code hinting or statement completions to the developer.

Starting with python 3.5 (September 2015), python allowed the developer to provide type annotation (typing hints) when defining functions and classes.

This additional information is ignored by the python interpreter, but is used by static type checker and IDE's to provide a better tooling experience for the user as well as code analysis and error detection.

As a result, there is a growing trend amongst software developer to practice type annotation.

```
def area_of_triangle(base: int, height: int) -> float:
    return base * height * 0.5

b, h = 5, 3
a = area_of_triangle(b, h)
print(f'A triangle of base {b} and height {h} will have an area of {a:.1f}')
A triangle of base 5 and height 3 will have an area of 7.5
```

Closure (Advance)

Functions that remember values from their enclosing scope.

```
def power(x):
    def foo(y):
        return y ** x

    return foo
power_to_2 = power(2)
```

Lecture 05

```
power_to_3 = power(3)

print(power_to_2(5))
print(power_to_3(4))
25
64
```

Parameters and Arguments

- Parameters: Variable name in the function definition
- Argument: Actual value passed when calling the function

```
def greet(name): # 'name' is a parameter
    print(f"Hello, {name}!")

greet("Alice") # "Alice" is an argument
Hello, Alice!
```

Parameter

A parameter is named entity in a function (or method) definition that specifies an argument (or in some cases, arguments) that the function can accept. There are five kinds of parameter:

1. Position-or-Keyword Parameters
2. Position-Only Parameters
3. Keyword-Only Parameters
4. Var-Positional (*args)
5. Var-Keyword (**kwargs)

Ordering the parameters

When working with multiple parameters, you should try to order them by how likely the caller is to change them.

Parameters that are more likely to remain the same across function calls go first, and parameters that more likely to change across function calls go last.

```
# 1. Position-or-Keyword Parameters
# These can be passed either positionally or as keyword arguments:
def greet(name, age):
    return f'Hello {name}, you are {age} years old'

# Both ways work:
greet('Alice', 25) # positional
greet(name='Alice', age=25) # keyword
greet('Alice', age=25) # mixed
'Hello Alice, you are 25 years old'

# 2. Position-Only Parameters
# Use / to mark parameters that can only be passed positionally:
def divide(numerator, denominator, /):
    return numerator / denominator
```

Lecture 05

```
# This works:
divide(10, 2)

# This would raise an error:
# divide(numerator=10, denominator=2) # TypeError!
5.0

# 3. Keyword-Only Parameters
# Use * to mark parameters that can only be passed as keywords:
def create_user(name, *, email, admin=False):
    return f'User: {name}, Email: {email}, Admin: {admin}'

# This works:
create_user('Bob', email='bob@example.com')
create_user('Bob', email='bob@example.com', admin=True)

# This would raise an error:
# create_user('Bob', 'bob@example.com') # TypeError!
'User: Bob, Email: bob@example.com, Admin: True'

# 4. Var-Positional (*args)
# Collects extra positional arguments into a tuple:
def sum_numbers(*numbers):
    return sum(numbers)

print(sum_numbers(1, 2, 3, 4, 5)) # Returns 15
print(sum_numbers())              # Returns 0
15
0

# 5. Var-Keyword (**kwargs)
# Collects extra keyword arguments into a dictionary:
def print_info(**info):
    for key, value in info.items():
        print(f'{key}: {value}')

print_info(name='Charlie', age=30, city='New York')

# Output:
# name: Charlie
# age: 30
# city: New York
name: Charlie
age: 30
city: New York

# Complete Example
# Here's a function using all parameter types:
def complex_function(pos_only, /, pos_or_kw, *args, kw_only, **kwargs):
    print(f'Position-only: {pos_only}')
    print(f'Position-or-keyword: {pos_or_kw}')
    print(f'Extra positional args: {args}')
    print(f'Keyword-only: {kw_only}')
    print(f'Extra keyword args: {kwargs}')

# Usage:
complex_function(1, 2, 3, 4, kw_only='required', extra='optional')
Position-only: 1
```


Lecture 05

Position-or-keyword: 2
Extra positional args: (3, 4)
Keyword-only: required
Extra keyword args: {'extra': 'optional'}

Recursive Function

Recursive functions are function that calls itself

```
def factorial(n: int) -> int:
    '''Calculate the factorial of a number.'''
    if n < 0:
        raise ValueError('Factorial is not defined for negative numbers.')
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
```

```
print(factorial(5)) # Output: 120
print(factorial(0)) # Output: 1
```

```
def reverse_string(start: str, end='') -> str:
    '''Reverse a given string.'''
    if len(start) == 0:
        return end
    return reverse_string(start[:-1], end + start[-1])
```

```
print(reverse_string('hello')) # Output: 'olleh'
print(reverse_string('world')) # Output: 'dlrow'
120
1
olleh
dlrow
```

Lambda Expression

A lambda expression is a way to create small anonymous function. This is useful because certain functions expects a function as an argument. (See the last few examples)

A lambda function may take zero or any number of arguments, but can only have one expression.

#lambda functions that takes a single argument

```
def square(x):
    return x * x
```

Lambda function equivalent

```
square_lambda = lambda x: x * x
```

Test the lambda function

```
print(square(5)) # Output: 25
print(square_lambda(5)) # Output: 25
25
25
```

#lambda functions that does not take any argument

Lecture 05

```
my_info = lambda : 'Narendra Pershad\nProgress Ave.\nToronto\nCanada'

# Test the lambda function
print(my_info())
Narendra Pershad
Progress Ave.
Toronto
Canada
#lambda functions that takes two arguments
my_name = lambda first, last: f'{first.capitalize()} {last.capitalize()}'

# Test the lambda function
print(my_name('narendra', 'pershad'))
Narendra Pershad
```

Function Naming

A sound practice when naming functions is to use a verb phrase.

```
# Good: Descriptive, verb-based names
def calculate_total_price(items, tax_rate):
    pass

def is_valid_email(email):
    pass

def get_user_by_id(user_id):
    pass

# Avoid: Vague or unclear names
def process(data): # What kind of processing?
    pass

def func1(x, y): # Not descriptive
    pass
```

Single Responsibility Principle

Each function should have one clear purpose

```
# Good: Separate concerns
def validate_email(email):
    '''Validate email format.'''
    pass

def send_email(to_address, subject, body):
    '''Send an email.'''
    pass

# Avoid: Doing too much in one function
def handle_user_registration(user_data):
    '''This does too much - validation, saving, emailing.'''
    pass
```

Lecture 05

Testing Your Functions

```
def celsius_to_fahrenheit(celsius):  
    '''Convert Celsius to Fahrenheit.'''  
    return (celsius * 9/5) + 32  
  
def test_celsius_to_fahrenheit():  
    '''Test the temperature conversion function.'''  
    # Test cases  
    assert celsius_to_fahrenheit(0) == 32, 'Freezing point test failed'  
    assert celsius_to_fahrenheit(100) == 212, 'Boiling point test failed'  
    assert celsius_to_fahrenheit(-40) == -40, 'Equal point test failed'  
    print('All temperature conversion tests passed!')  
  
# Run tests  
test_celsius_to_fahrenheit()  
All temperature conversion tests passed!
```

Summary

Functions are essential building blocks in Python programming that help you:

- Organize code into reusable, logical units
- Reduce repetition and improve maintainability
- Break complex problems into manageable pieces
- Create cleaner, more readable programs

Benefits of Using Functions

- Maintainability: Easier to update and debug
- Reusability: Write once, use multiple times
- Readability: Self-documenting code structure
- Testing: Isolated functionality for better testing
- Collaboration: Team members can work on different functions

Key concepts to remember:

- Functions can take any number of parameters (including zero)
- Functions can return values or perform actions
- Variable scope determines where variables can be accessed
- Type annotations and docstrings improve code quality
- Lambda functions provide a way to create simple, inline functions
- Recursive functions can solve problems by breaking them into smaller subproblems